



CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Weeks 3–4 • Day-by-Day Companion

Beginner • Python • PowerShell on Windows • 2 hrs/day • modern API patterns

SCOPE	Days 15–28 of 112 • Tool use + MCP
FOCUS	Agent loop • Strict tools • FastMCP servers • Multi-server clients
ACADEMY	Tool use with Claude • Model Context Protocol modules
SHELL	PowerShell on Windows • modern API patterns throughout
NEXT	Week 5 starts Portfolio Project 1 (Structured Data Extractor)

DAY-BY-DAY COMPANION • WEEKS 3–4

TOC

14 Days

15	Week 3 • Monday — Your First Tool
16	Week 3 • Tuesday — Multi-Turn Tool Conversations
17	Week 3 • Wednesday — Tool Design — Descriptions as Prompts
18	Week 3 • Thursday — Tool Use Meets Structured Outputs
19	Week 3 • Friday — Server Tools — Web Search
20	Week 3 • Saturday — Error Handling & Tool Caching
21	Week 3 • Sunday — Week 3 Review
22	Week 4 • Monday — MCP — Concepts & Primitives
23	Week 4 • Tuesday — Your First MCP Server
24	Week 4 • Wednesday — Resources & Prompts
25	Week 4 • Thursday — Transports — Stdio vs Streamable HTTP
26	Week 4 • Friday — Connecting Claude to MCP — Programmatic Client
27	Week 4 • Saturday — Multi-Server & Production Concerns
28	Week 4 • Sunday — Week 4 Review & Project 1 Prep

Each day pairs a hands-on lab with the relevant Anthropic Academy lessons. Code patterns use the modern API throughout (no `prefill`; `messages.parse()` for structured output; `strict=True` on tools). MCP module lesson names not screenshot-verified — send me a screenshot if Week 4 mappings look off.

WEEK 03

Monday

Day 15 of 112

Your First Tool

INTENT Stop watching Claude generate text. Start letting Claude take actions. Build a single-tool agent and walk through the loop manually.

00-10 Warm-up

Week 3 is where the course gets interesting. So far Claude has only generated text. Starting today, Claude can *call functions* — fetch the weather, query your database, send an email — and respond using the results.

This is the foundation of every agent you'll build for the rest of the plan. Take it slow today. The loop matters.

10-40 Theory — how tool use works

Read [the tool use overview](#) carefully. The mental model:

1. **You define tools** — name, description, JSON schema for inputs. Pass them via the `tools=[]` parameter.
2. **Claude decides whether to call one**, based on the user's request. If yes, the response contains a `tool_use` block instead of (or in addition to) text.
3. **Your code executes the tool** — runs the function with the args Claude provided.
4. **You send the result back** as a `tool_result` block in a new user message.
5. **Claude continues the conversation** — usually generates a final text response that incorporates the tool's output.

Notice the loop: each turn might trigger another tool call. You decide when to exit (usually when `response.stop_reason == "end_turn"`).

Two roles in the loop:

- **Claude** — picks the tool, fills in the arguments, interprets the result. It cannot execute the tool itself.
- **You** — provide the tool definition, run the actual function, return the result. Without you, the loop stops.

+30 ▶Watch on Anthropic Academy

From the **Tool use with Claude** module ([Skilljar](#)):

- **Lesson** · Introducing tool use
- **Lesson** · Project overview — sets up the running example used through this module
- **Lesson** · Tool functions — defining the Python functions Claude will call

~20 minutes. Watch the module's project overview carefully — the example carries through ~10 lessons and your understanding compounds.

40-110 Lab — single-tool agent

Set up Week 3's folder:

```
cd C:\Users\\Desktop\cca-prep
mkdir labs\week-03
```

Create `labs\week-03\first_tool.py`:

```
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

# Step 1: Define the tool. Description matters – it's a prompt for Claude.
TOOLS = [
    {
        "name": "get_weather",
        "description": (
            "Get the current weather for a given location. "
            "Use this whenever the user asks about weather conditions, temperature, "
            "or whether it's raining/snowing/sunny in a specific city."
        ),
        "input_schema": {
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": "City and country, e.g. \"Bratislava, Slovakia\" or \"Tokyo, Japan\"",
                },
            },
            "required": ["location"],
        },
        "strict": True, # Guarantees Claude's arguments match the schema exactly
    }
]

# Step 2: The actual function that runs when Claude calls the tool.
def get_weather(location: str) -> dict:
    # Stubbed – pretend we called a real weather API.
    fake_data = {
        "Bratislava, Slovakia": {"temp_c": 14, "conditions": "partly cloudy"},
        "Tokyo, Japan": {"temp_c": 26, "conditions": "sunny"},
    }
    return fake_data.get(location, {"temp_c": 20, "conditions": "unknown"})

# Step 3: Run the conversation loop.
messages = [{"role": "user", "content": "What's the weather in Bratislava right now?"}]

while True:
```

```

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    tools=TOOLS,
    messages=messages,
)

print(f"
stop_reason: {response.stop_reason}")

if response.stop_reason == "end_turn":
    # Claude is done. Print its final text answer.
    for block in response.content:
        if block.type == "text":
            print(f"Claude: {block.text}")
    break

if response.stop_reason == "tool_use":
    # Claude wants to call a tool. Append its turn to history first.
    messages.append({"role": "assistant", "content": response.content})

    # Find each tool_use block, execute it, build tool_result blocks.
    tool_results = []
    for block in response.content:
        if block.type == "tool_use":
            print(f"Tool call: {block.name}({block.input})")
            if block.name == "get_weather":
                result = get_weather(block.input["location"])
            else:
                result = {"error": "unknown tool"}

            tool_results.append({
                "type": "tool_result",
                "tool_use_id": block.id,
                "content": str(result),
            })

    # Send results back as a new user message.
    messages.append({"role": "user", "content": tool_results})
    # Loop continues – Claude will see the result and respond.

```

Run:

```
uv run python labs\week-03\first_tool.py
```

You should see:

1. First iteration: `stop_reason: tool_use`, Claude calls `get_weather(location="Bratislava, Slovakia")`
2. Your code executes the function, returns the result
3. Second iteration: `stop_reason: end_turn`, Claude says something like "It's 14°C and partly cloudy in Bratislava right now."

Now break it deliberately: change the user message to "What's 2+2?". Re-run. Claude should NOT call the tool — its description doesn't match a math question. `stop_reason` goes straight to `end_turn`. That's how tool descriptions act as a prompt: Claude only calls a tool when its description matches the user's intent.

110-120 **Capture**

Create `notes\03-tool-use.md` and write 3-5 sentences answering: "What goes in the `tools` parameter, what does Claude return when it wants to call one, and how do you send results back?" Commit.

```
git add .  
git commit -m "day 15: first tool use"  
git push
```

END OF DAY `first tool.py` runs end-to-end. You can explain the 5-step tool loop without notes.

WEEK 03

Tuesday

Day 16 of 112

Multi-Turn Tool Conversations

INTENT Generalize yesterday's hand-coded loop into a reusable agent function that handles multiple tool calls per conversation.

00-10 Warm-up

Yesterday's loop handled one tool call. Real conversations are messier — a user might ask "What's the weather in Tokyo and Paris?", which Claude can answer with two tool calls in one turn, OR with sequential calls across two turns. Today you build a loop that handles both.

10-40 Theory — the canonical agent loop

The pattern you wrote yesterday is the canonical agent loop. Every Claude agent — including the production support bot you'll build in Project 2 — uses the same skeleton:

```
messages = [user_request]
while True:
    response = client.messages.create(model=..., tools=tools, messages=messages)

    if response.stop_reason == "end_turn":
        return final_text(response)

    if response.stop_reason == "tool_use":
        messages.append({"role": "assistant", "content": response.content})
        tool_results = execute_all_tool_calls(response)
        messages.append({"role": "user", "content": tool_results})
        continue

    # Edge cases: "max_tokens" (response truncated), "stop_sequence", etc.
    raise RuntimeError(f"Unexpected stop_reason: {response.stop_reason}")
```

Three production concerns that show up on the exam:

- **Iteration cap.** A pathological prompt can loop calling tools forever. Add `max_iterations=10` and raise if exceeded.
- **Multiple tool calls per turn.** One `stop_reason=tool_use` response can contain several `tool_use` blocks. You must execute all of them and bundle their results into one user message.
- **Always append the full assistant message before sending results.** Claude must see its own prior `tool_use` blocks. Skipping this breaks the conversation.

+30

►Watch on Anthropic Academy

From the **Tool use with Claude** module ([Skilljar](#)):

- Lesson • Handling message blocks — exactly today's loop topic
- Lesson • Sending tool results
- Lesson • Multi-turn conversations with tools
- Lesson • Implementing multiple turns

~25 minutes total. This is the heaviest theory day of Week 3 — these four lessons together cover the agent loop end-to-end.

40-110 Lab — reusable agent loop

Create `labs\week-03\agent_loop.py`:

```

from typing import Callable
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

TOOLS = [
    {
        "name": "get_weather",
        "description": "Get current weather for a city. Use for any weather/temperature question.",
        "input_schema": {
            "type": "object",
            "properties": {"location": {"type": "string"}},
            "required": ["location"],
        },
        "strict": True,
    },
    {
        "name": "convert_currency",
        "description": "Convert an amount from one currency to another. Use for any FX conversion.",
        "input_schema": {
            "type": "object",
            "properties": {
                "amount": {"type": "number"},
                "from_currency": {"type": "string", "description": "ISO code, e.g. USD"},
                "to_currency": {"type": "string", "description": "ISO code, e.g. EUR"},
            },
            "required": ["amount", "from_currency", "to_currency"],
        },
        "strict": True,
    },
]

# Tool implementations (stubs)
def get_weather(location: str) -> dict:
    return {"location": location, "temp_c": 18, "conditions": "clear"}

def convert_currency(amount: float, from_currency: str, to_currency: str) -> dict:
    fake_rates = {("USD", "EUR"): 0.92, ("EUR", "USD"): 1.09, ("USD", "JPY"): 150.0}
    rate = fake_rates.get((from_currency, to_currency), 1.0)
    return {"amount": round(amount * rate, 2), "from": from_currency, "to": to_currency}

TOOL_REGISTRY: dict[str, Callable] = {
    "get_weather": get_weather,

```

```

    "convert_currency": convert_currency,
}

def run_agent(user_message: str, max_iterations: int = 10) -> str:
    messages = [{"role": "user", "content": user_message}]

    for iteration in range(max_iterations):
        response = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=1024,
            tools=TOOLS,
            messages=messages,
        )

        print(f"  iter {iteration+1}: stop_reason={response.stop_reason}")

        if response.stop_reason == "end_turn":
            # Extract final text
            for block in response.content:
                if block.type == "text":
                    return block.text
            return "(no text response)"

        if response.stop_reason == "tool_use":
            messages.append({"role": "assistant", "content": response.content})

            tool_results = []
            for block in response.content:
                if block.type == "tool_use":
                    print(f"    -> calling {block.name}({block.input})")
                    fn = TOOL_REGISTRY.get(block.name)
                    if not fn:
                        result = {"error": f"unknown tool: {block.name}"}
                        is_error = True
                    else:
                        try:
                            result = fn(**block.input)
                            is_error = False
                        except Exception as e:
                            result = {"error": str(e)}
                            is_error = True

                    tool_results.append({
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": str(result),
                        "is_error": is_error,
                    })

            messages.append({"role": "user", "content": tool_results})
            continue

        raise RuntimeError(f"Unexpected stop_reason: {response.stop_reason}")

```

```
raise RuntimeError(f"Agent exceeded {max_iterations} iterations")

# Try three increasingly complex prompts
for prompt in [
    "What's the weather in Tokyo?",
    "Convert 100 USD to EUR.",
    "Compare the weather in Tokyo and Bratislava, and tell me how much 50 EUR is in JPY.",
]:
    print(f"
User: {prompt}")
    answer = run_agent(prompt)
    print(f"Claude: {answer}")
```

Run it:

```
uv run python labs\week-03\agent_loop.py
```

The third prompt is the interesting one. Claude may handle it as:

- **Parallel:** one iteration with three `tool_use` blocks (weather Tokyo, weather Bratislava, convert EUR→JPY)
- **Sequential:** three iterations, one tool call each
- **Mixed:** two parallel calls in iteration 1, one more in iteration 2

Your loop handles all three correctly — that's the point of building it once and reusing.

110-120 Capture

Update your notes: copy the canonical agent-loop pseudocode in your own words. **This is exam-critical** — you'll see scenario questions asking what the loop should look like, and verbatim memorization helps. Commit.

END OF DAY A reusable `run_agent()` function that handles parallel and sequential tool calls correctly.

WEEK 03

Wednesday

Day 17 of 112

Tool Design — Descriptions as Prompts

INTENT Internalize the single highest-leverage skill in tool use: writing tool descriptions Claude can actually act on. Most exam-relevant skill of Week 3.

00-10 Warm-up

Reopen `first_tool.py` from Monday. Read the tool description out loud. Ask yourself: "If I knew nothing about this codebase, could I tell from this description alone (a) when to call this tool, (b) when NOT to call it, (c) what each argument means?"

If yes to all three — exam-grade. If no on any — that's today's lesson.

10-40 Theory — what makes a description production-grade

Tool descriptions are prompts. Claude reads them on every turn to decide what to call. A bad description is the single most common reason agents misbehave on the exam (and in production).

Five properties of a production-grade tool description:

1. **States what the tool does, in one sentence.** "Fetches the customer record by customer_id from the CRM."
2. **States when to use it.** "Use whenever the user mentions a specific customer, order, or account question."
3. **States when NOT to use it.** "Do not use for general policy questions — use `get_policy` instead."
4. **Specifies constraints/preconditions.** "Requires a customer_id starting with `CUST-` . If you don't have one, call `search_customer` first."
5. **Includes a one-line example.** "Example: `get_customer(customer_id='CUST-12345')`."

For **arguments**, the same rule applies — every property in the JSON schema needs its own `description` field. "user_id" is not enough. "Numeric internal user ID (not the email or display name)" is.

Strict mode (`"strict": True` on the tool) guarantees the arguments match the schema exactly — Claude can't invent a field or pass a string where you asked for a number. Use it on every tool you write from now on. Read [the strict tool use docs](#).

+30 ▶Watch on Anthropic Academy

From the **Tool use with Claude** module ([Skilljar](#)):

- Lesson • Tool schemas
- Lesson • Using multiple tools
- Lesson • Fine grained tool calling — preview of strict mode mechanics

~20 minutes. The *Tool schemas* lesson directly covers what you're practicing today.

40-110 Lab — rewrite your tools to exam quality

Take yesterday's `agent_loop.py` and rewrite the tool definitions to be production-grade. Save as `labs\week-03\polished_tools.py`:

```
TOOLS = [
    {
        "name": "get_weather",
        "description": (
            "Get the current weather conditions and temperature for a specific city. "
            "Use this whenever the user asks about weather, temperature, rain, snow, "
            "or whether they need a jacket/umbrella in a named location. "
            "Do NOT use for forecasts beyond today – only current conditions are available. "
            "Example: get_weather(location=\"Bratislava, Slovakia\")"
        ),
        "input_schema": {
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": (
                        "City and country in 'City, Country' format. "
                        "Country name in English, not ISO code. "
                        "If only a city is given, infer the country (e.g. 'Tokyo' -> 'Tokyo, "
                        "Japan'). "
                        "If the city is ambiguous (e.g. 'Springfield'), ask the user to "
                        "clarify "
                        "rather than guessing."
                    )
                },
            },
            "required": ["location"],
        },
        "strict": True,
    },
    {
        "name": "convert_currency",
        "description": (
            "Convert a monetary amount from one currency to another using current exchange "
            "rates. "
            "Use whenever the user asks 'how much is X in Y?' or wants a price translated. "
            "Do NOT use for historical rates – only current rates are available. "
            "Returns a dict with the converted amount and the rate used. "
            "Example: convert_currency(amount=100, from_currency=\"USD\", "
            "to_currency=\"EUR\")"
        ),
        "input_schema": {
            "type": "object",
            "properties": {
                "amount": {
                    "type": "number",
                    "description": "Amount to convert, as a positive number.",
                },
                "from_currency": {
                    "type": "string",
```

```
        "description": "Source currency as ISO 4217 code, e.g. USD, EUR, JPY, GBP.  
Three uppercase letters.",  
      },  
      "to_currency": {  
        "type": "string",  
        "description": "Target currency as ISO 4217 code, same format.",  
      },  
    },  
    "required": ["amount", "from_currency", "to_currency"],  
  },  
  "strict": True,  
},  
]
```

Now run a calibration test. Use the same loop from yesterday with these new tools and try ambiguous prompts:

1. "What's the weather in Springfield?" — should ask for clarification (your description says so)
2. "How much was 100 USD in EUR last year?" — should refuse / state limitation (description forbids historical)
3. "Convert 50 to dollars" — should ask for source currency (required by schema + strict mode)

Compare against yesterday's vague descriptions — the difference is striking. **Tool descriptions are the most exam-tested aspect of tool use** because they're the most common source of agent failures in production. Memorize the five properties.

110-120 Capture

Note the 5 properties of a production-grade tool description in your own words, with one example per property. This single page of notes will earn you points on the exam. Commit.

END OF DAY Polished, strict-mode tools with rich descriptions. Behaves correctly on ambiguous prompts.

WEEK 03

Thursday

Day 18 of 112

Tool Use Meets Structured Outputs

INTENT Wire up the two patterns you now know — tool calling and structured outputs — into a single agent that extracts validated data through tools.

00-10 Warm-up

You now know two ways to get reliable structured output from Claude:

- **Structured Outputs** (Week 2): `messages.parse(output_format=PydanticModel)` — for the final response
- **Tool use with strict mode** (this week): `tools=[{..., "strict": True}]` — for tool arguments mid-loop

Today: when to use which.

10-40 Theory — picking the right pattern

Decision rule:

- **Final answer needs to be a typed object?** → Structured Outputs. `messages.parse()` with `output_format=YourModel`.
- **Claude needs to invoke a function or call an API?** → Tool use with `strict: True`. The strict flag does for tool arguments what structured outputs does for the final response.
- **Both?** Yes, frequently. An agent calls tools to fetch data, then returns a typed object as its final answer. See today's lab.

Common exam trap: a scenario shows Claude returning JSON that contains tool-call-like fields ("function": "get_user", "args": Ellipsis). The answer is almost always "use proper tool use with strict mode, don't fake function calls via structured outputs." Tools have their own first-class support — use it.

+30 ▶ Watch on Anthropic Academy

Today's pairing isn't a single course lesson — it's the synthesis of two threads. Re-watch (or skim) one lesson from each module:

- **Lesson** • Structured data from *Accessing Claude with the API* — recall the structured-outputs pattern
- **Lesson** • Fine grained tool calling from *Tool use with Claude* — the strict-mode mechanics

If you skipped *Fine grained tool calling* earlier this week, watch it today — it's where strict mode is taught.

40-110 Lab — research agent with typed output

Build a research agent that calls tools to gather data, then returns a typed `CityReport` .

Create `labs\week-03\research_agent.py` :

```
from pydantic import BaseModel
from typing import Literal
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

# Tool definitions (same pattern as Monday)
TOOLS = [
    {
        "name": "get_population",
        "description": "Get the current population of a city. Returns the number of residents.",
        "input_schema": {
            "type": "object",
            "properties": {"city": {"type": "string", "description": "City name in English"}},
            "required": ["city"],
        },
        "strict": True,
    },
    {
        "name": "get_climate",
        "description": "Get the climate classification (Köppen) and average temperature for a city.",
        "input_schema": {
            "type": "object",
            "properties": {"city": {"type": "string", "description": "City name in English"}},
            "required": ["city"],
        },
        "strict": True,
    },
]

# Stub implementations
def get_population(city: str) -> dict:
    return {"city": city, "population": {"Tokyo": 139600000, "Bratislava": 475500}.get(city, 1000000)}

def get_climate(city: str) -> dict:
    return {"city": city, "koppen": {"Tokyo": "Cfa", "Bratislava": "Dfb"}.get(city, "Cfb"), "avg_temp_c": 14}

TOOL_REGISTRY = {"get_population": get_population, "get_climate": get_climate}
```

```

# Final output schema
class CityReport(BaseModel):
    city: str
    population: int
    climate_classification: str
    avg_temperature_c: float
    size_category: Literal["small", "medium", "large", "megacity"]

def gather_facts(city: str) -> list[dict]:
    """Run the tool loop to collect raw facts."""
    messages = [{"role": "user", "content": (
        f"Use the available tools to gather facts about {city}. "
        f"Call get_population AND get_climate. Then summarize what you learned."
    )}]
    facts_collected = []

    for _ in range(10):
        response = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=1024,
            tools=TOOLS,
            messages=messages,
        )

        if response.stop_reason == "end_turn":
            return facts_collected

        if response.stop_reason == "tool_use":
            messages.append({"role": "assistant", "content": response.content})
            tool_results = []
            for block in response.content:
                if block.type == "tool_use":
                    result = TOOL_REGISTRY[block.name](**block.input)
                    facts_collected.append(result)
                    tool_results.append({
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": str(result),
                    })
            messages.append({"role": "user", "content": tool_results})

    return facts_collected

def build_report(city: str) -> CityReport:
    """Tool-loop to gather; structured output to format."""
    facts = gather_facts(city)

    # Second call: just structured output, no tools
    response = client.messages.parse(
        model="claude-sonnet-4-6",
        max_tokens=512,
        system="Produce a CityReport from the collected facts.",
    )

```

```
        messages=[{"role": "user", "content": f"City: {city}"},
        Facts: {facts}}}],
        output_format=CityReport,
    )
    return response.parsed_output

# Run it
for city in ["Tokyo", "Bratislava"]:
    report = build_report(city)
    print(f"
{report.model_dump_json(indent=2)}")
```

This is the canonical production pattern: **tool loop for data gathering, structured outputs for the final response**. You'll see this exact architecture in Project 2 (Customer Support Agent) and Project 3 (Multi-Agent Research).

110-120 Capture

Note the decision rule (structured outputs vs strict tool use vs both) and one example of each. Commit.

END OF DAY An agent that gathers facts via tools and returns a Pydantic-validated final report.

WEEK 03

Friday

Day 19 of 112

Server Tools — Web Search

INTENT Use an Anthropic-hosted tool (no execution code needed on your side). The `web_search` tool is the gateway to real-world data.

00-10 Warm-up

So far every tool you've defined required you to write the implementation. Today: a tool where Anthropic runs the implementation. You just declare it and they handle the rest.

This is called a **server tool**. Three exist today: `web_search`, `web_fetch`, and `code_execution`. Today you use the first.

10-40 Theory — client tools vs server tools

Client tools (what you've built all week) — run in YOUR code. Claude returns a `tool_use` block, you execute, you send back a `tool_result`. Examples: `get_weather`, `process_refund`, anything calling your backend.

Server tools — run on Anthropic's infrastructure. You declare them in the `tools` array but you don't write an implementation. Claude calls them, executes them server-side, and the results appear directly in the response. No tool loop iteration needed on your side for server tools.

The three Anthropic server tools currently:

- `web_search` — searches the web, returns ranked results with snippets. Read [the web search docs](#).
- `web_fetch` — fetches a specific URL and returns its content.
- `code_execution` — runs Python in a sandbox. Useful for math, data analysis, file processing.

Why this matters for the exam: Project 3 (Multi-Agent Research Pipeline) leans heavily on `web_search`. Knowing it's a server tool (no local implementation) saves you from writing your own search infrastructure.

Server tools are billed differently — each tool has its own per-call cost on top of token costs. Check [pricing](#) before running batch jobs.

+30 ▶Watch on Anthropic Academy

From the **Tool use with Claude** module ([Skilljar](#)):

- Lesson • The web search tool — direct match for today's lab
- Lesson • The text edit tool — different server tool, complements your understanding

~15 minutes.

40-110 Lab — research with web search

Create `labs\week-03\web_search_agent.py`:

```
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

# Note: the type identifier includes a date version. Check the docs for the current one.
TOOLS = [
    {
        "type": "web_search_20250305",
        "name": "web_search",
        "max_uses": 3, # cap how many searches Claude can run for one query
    }
]

def research(question: str) -> str:
    messages = [{"role": "user", "content": question}]

    # The server tool loop is similar to client tools, but you don't execute anything.
    # Claude calls the tool, Anthropic runs it, results come back directly.
    for _ in range(5):
        response = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=2048,
            tools=TOOLS,
            messages=messages,
        )

        # Check what happened
        for block in response.content:
            if block.type == "server_tool_use":
                print(f" Claude searched: {block.input.get(\"query\", \"?\")}")
            elif block.type == "web_search_tool_result":
                count = len(block.content) if isinstance(block.content, list) else 1
                print(f" Got {count} search results")

        if response.stop_reason == "end_turn":
            for block in response.content:
                if block.type == "text":
                    return block.text
            return "(no text)"

    # For server tools, the assistant turn already contains the results.
    # We just keep going.
    messages.append({"role": "assistant", "content": response.content})

    return "(exceeded iterations)"
```

```
for question in [
    "What were the three biggest AI announcements at NeurIPS 2025?",
    "How much does the Anthropic CCA-Foundations exam cost as of today?",
]:
    print(f"
Q: {question}")
    print(f"A: {research(question)}
")
```

Run:

```
uv run python labs\week-03\web_search_agent.py
```

Notice the key differences from client tools:

- No implementation function in your code.
- No `tool_result` block to send back. Results are already in the assistant message.
- `server_tool_use` and `web_search_tool_result` blocks appear, not `tool_use`.
- `max_uses` caps spend — server tools cost per call.

Exam note: if a scenario asks "how do you give an agent access to the web?", the answer is `web_search` server tool, not "build a custom client tool that calls Google's API." Anthropic provides the canonical version.

110-120 Capture

Note the difference between client tools and server tools — the test of understanding is whether you can explain in one sentence why `web_search` doesn't need a local implementation. Commit.

END OF DAY Working web-search agent. You can explain server tools vs client tools.

WEEK 03

Saturday

Day 20 of 112

Error Handling & Tool Caching

INTENT *Make tool-using agents production-grade. Handle tool failures gracefully. Use prompt caching with tools — Anthropic's recommended cost optimization for tool-heavy agents.*

00-10 Warm-up

What happens when `get_customer("CUST-99999")` returns 'customer not found'? Does the agent loop forever? Does it crash? Does it gracefully escalate? Today: make it the third one.

10-40 Theory — three error categories

Tool errors come in three categories. Production agents handle each differently:

1. **Expected business errors** — "customer not found", "refund window expired", "out of stock". Return them as normal `tool_result` blocks with `is_error: true`. Claude sees the error message and responds appropriately (apologizes, suggests alternatives, escalates). *Do not raise Python exceptions for these.*
2. **Transient failures** — network timeouts, 429 rate limits, brief 5xx errors. Wrap the tool call in retry-with-exponential-backoff inside the tool implementation. If retries exhaust, return `is_error: true` to Claude.
3. **Bugs / unexpected exceptions** — your code has a real problem. Catch broadly, log, return `is_error: true` with a generic error message (NEVER expose stack traces to Claude — it may leak internal info to the user). Alert your error tracking system.

Crucial pattern: never let an exception bubble out of your tool execution into the agent loop. The loop crashes and the user gets nothing. Always wrap in try/except, always return a `tool_result` block.

Prompt caching with tools: tool definitions are static across calls — perfect for caching. Add `cache_control: {"type": "ephemeral"}` to the LAST tool in your tools array. Everything from the start of tools through that cache point gets cached. Cuts cost on tool-heavy workflows by 5-10x.

+30 ▶ Watch on Anthropic Academy

This is a synthesis day — the relevant material is split across modules. Either:

- Re-watch the caching trio from *Features of Claude*: [Lesson](#) • Prompt caching, [Lesson](#) • Rules of prompt caching, [Lesson](#) • Prompt caching in action — and apply mentally to tools instead of system prompts.
- Or read [the prompt caching docs](#) directly, focusing on the section on caching tools.

40-110 Lab — production-grade error handling + cached tools

Create `labs\week-03\production_agent.py`:

```
import logging
import time
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
client = Anthropic()

logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
log = logging.getLogger(__name__)

# Tools array with cache_control on the last tool – caches all tool defs
TOOLS = [
    {
        "name": "get_customer",
        "description": "Look up a customer by ID. Returns customer record or 'not found'.",
        "input_schema": {
            "type": "object",
            "properties": {"customer_id": {"type": "string"}},
            "required": ["customer_id"],
        },
        "strict": True,
    },
    {
        "name": "get_order",
        "description": "Look up an order by order ID. Returns order details or 'not found'.",
        "input_schema": {
            "type": "object",
            "properties": {"order_id": {"type": "string"}},
            "required": ["order_id"],
        },
        "strict": True,
        "cache_control": {"type": "ephemeral"}, # cache from beginning through this tool
    },
]

# Tool implementations with proper error handling
FAKE_CUSTOMERS = {"CUST-001": {"name": "Acme Corp", "tier": "enterprise"}}
FAKE_ORDERS = {"ORD-100": {"customer_id": "CUST-001", "total": 250.0, "status": "delivered"}}

def safe_call(fn, **kwargs) -> tuple[str, bool]:
    """Returns (content_str, is_error). Never raises."""
    try:
        result = fn(**kwargs)
        return (str(result), False)
    except KeyError as e:
        return (f"Not found: {e}", True)
    except Exception as e:
```

```

        log.exception("Tool execution failed")
        # Generic error to Claude; details to logs
        return ("Internal error. Try again or escalate.", True)

def get_customer(customer_id: str) -> dict:
    if customer_id not in FAKE_CUSTOMERS:
        raise KeyError(customer_id)
    return FAKE_CUSTOMERS[customer_id]

def get_order(order_id: str) -> dict:
    if order_id not in FAKE_ORDERS:
        raise KeyError(order_id)
    return FAKE_ORDERS[order_id]

TOOL_REGISTRY = {"get_customer": get_customer, "get_order": get_order}

def run_agent(user_msg: str, max_iter: int = 10) -> dict:
    messages = [{"role": "user", "content": user_msg}]
    total_cache_create = 0
    total_cache_read = 0

    for iteration in range(max_iter):
        response = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=1024,
            tools=TOOLS,
            messages=messages,
        )

        # Track cache stats
        usage = response.usage
        total_cache_create += getattr(usage, "cache_creation_input_tokens", 0) or 0
        total_cache_read += getattr(usage, "cache_read_input_tokens", 0) or 0

        if response.stop_reason == "end_turn":
            text = next((b.text for b in response.content if b.type == "text"), "")
            return {
                "answer": text,
                "iterations": iteration + 1,
                "cache_create_tokens": total_cache_create,
                "cache_read_tokens": total_cache_read,
            }

        if response.stop_reason == "tool_use":
            messages.append({"role": "assistant", "content": response.content})
            tool_results = []
            for block in response.content:
                if block.type == "tool_use":
                    fn = TOOL_REGISTRY.get(block.name)
                    if not fn:
                        content, is_error = "Unknown tool.", True

```

```

        else:
            content, is_error = safe_call(fn, **block.input)
        tool_results.append({
            "type": "tool_result",
            "tool_use_id": block.id,
            "content": content,
            "is_error": is_error,
        })
    messages.append({"role": "user", "content": tool_results})

    raise RuntimeError(f"exceeded {max_iter} iterations")

# Two runs back-to-back – second should hit the tool cache
for prompt in [
    "Look up customer CUST-001 and tell me about their account.",
    "Look up order ORD-100 and tell me when it was delivered.",
    "Look up customer CUST-99999 – they're asking about their order.",
]:
    print(f"
Q: {prompt}")
    result = run_agent(prompt)
    print(f"A: {result[\"answer\"]}")
    print(f"    [iters: {result[\"iterations\"]}, "
          f"cache_create: {result[\"cache_create_tokens\"]}, "
          f"cache_read: {result[\"cache_read_tokens\"]}"])
```

Run it. Observe two things:

1. The third prompt (CUST-99999, doesn't exist) — Claude calls the tool, gets `is_error: true` with "Not found", and responds gracefully (apologizes, asks for the right ID). No crash.
2. Cache stats: first run shows `cache_create` tokens (paying to write the cache). Subsequent runs show `cache_read` tokens (using the cache). Token cost drops sharply.

110-120 Capture

Note the 3 error categories and how each is handled. This will appear on the exam as scenario questions. Commit.

END OF DAY Production-grade error handling. Cached tools. Cache token savings visible in stats.

WEEK 03

Sunday

Day 21 of 112

Week 3 Review

INTENT Lock in tool use. Take the official Skilljar quiz. Prep for MCP (Week 4).

00–10 Warm-up

Three weeks in. You can now build an agent that uses tools, handles errors, caches efficiently, and returns typed output. That's already more than most exam candidates can do.

10–60 Review — re-run every Week 3 lab

```
uv run python labs\week-03\first_tool.py
uv run python labs\week-03\agent_loop.py
uv run python labs\week-03\polished_tools.py
uv run python labs\week-03\research_agent.py
uv run python labs\week-03\web_search_agent.py
uv run python labs\week-03\production_agent.py
```

For each, ask: "Could I rewrite this from scratch tomorrow without notes?" If no on any, re-read the notes and redo the lab.

+30 ▶ Watch on Anthropic Academy

Formal self-test for Week 3 ([Skilljar](#) • Tool use with Claude module):

■ **Quiz** • Quiz on tool use with Claude

Aim 80%+. The quiz covers tool definitions, the loop, multi-turn handling, parallel use, server tools — all of which you've practiced. If you score below, identify which lesson area you missed and re-watch.

60–110 Self-check & Week 4 prep

Week 3 self-check question (from the master plan): "What makes a tool description 'production-grade'? Give five properties."

Write the answer in `notes\weekly-reviews.md`. No looking at your labs. Then write:

1. Hardest moment this week was...
2. The pattern I want to internalize before Project 2 is...
3. One question I'd ask if I could...

Prep for Week 4:

```
mkdir labs\week-04
```

Week 4 is **Model Context Protocol (MCP)** — Anthropic's open standard for tool sharing. Instead of every developer writing their own `get_customer` tool, MCP lets you ship a server once and any compatible client (Claude Desktop, Cursor, Claude Code, your own agent) can use it.

You'll build your first MCP server on Tuesday and connect to it programmatically by Saturday. Restful Sunday tonight.

110-120 **Capture**

```
git add .  
git commit -m "week 3 complete: tool use"  
git push
```

Close the laptop. Take the rest of Sunday off.

END OF DAY Week 3 complete. Tool-use quiz passed. You can build agents that call tools and handle errors.

WEEK 04

Monday

Day 22 of 112

MCP — Concepts & Primitives

INTENT Understand WHY MCP exists, the five primitives, and the two transport options. Read before coding.

00-10 Warm-up

Week 3 taught you tools that live in YOUR code. Week 4 teaches tools that live in OTHER PEOPLE'S servers and that Claude can use through a standard protocol. The pattern is the same; the deployment story is completely different.

10-40 Theory — what is MCP, really

Read modelcontextprotocol.io — the intro and primitives sections. ~20 minutes.

The problem MCP solves: Without MCP, every tool integration is custom code per LLM provider. You write a "Salesforce tool" for Claude with its tool schema, then rewrite it for OpenAI with their schema, then again for Gemini. MCP standardizes the format so a single MCP server works across all compatible clients.

The five primitives — memorize cold for the exam:

1. **Tools** — functions the LLM can call. Same concept as Week 3 tools, just exposed over the protocol.
2. **Resources** — data the LLM can read. Think files, database rows, API responses. Identified by URIs (e.g. `file:///notes/meeting-2026-06-01.md`).
3. **Prompts** — reusable prompt templates the server provides to the client. Useful for "best practices" prompts a vendor wants to ship.
4. **Sampling** — server can ask the client (LLM) to do completions on its behalf. Rare; advanced.
5. **Roots** — directories/URIs the server is allowed to operate within. Security boundary.

The two transports:

- **stdio** — server runs as a subprocess of the client, communicates over stdin/stdout. Used for local servers. Simple, no auth needed.
- **streamable-http** — server runs over HTTP with SSE. Used for remote/networked servers. Requires auth (OAuth 2.1 typical).

You'll write a stdio server tomorrow and connect it to Claude Desktop by Wednesday.

+30 ▶Watch on Anthropic Academy

From the **Model Context Protocol** module ([Skilljar](#)). I haven't verified the exact lesson names against the live Skilljar UI — expand the module in your sidebar and watch the introductory lessons (typically the first 2-3, covering concepts and primitives).

Honest note: if the lesson names are very different from what I'd guess, ping me with a screenshot and I'll update Week 4's mapping the way we did for Weeks 1-2.

~20-30 minutes whatever the lessons are titled.

40-110 **Lab — install MCP, run the example server**

Add MCP dependencies and verify the install:

```
uv add mcp
uv run python -c "from mcp.server.fastmcp import FastMCP; print(\"MCP OK\")"
```

Now run a pre-built MCP server and connect to it. The `@modelcontextprotocol/server-everything` server (Node-based) exposes all five primitives — great for exploration.

From PowerShell:

```
npx -y @modelcontextprotocol/server-everything
```

Wait for it to start. It speaks stdio, so you won't see output unless you send messages.

Open **Claude Desktop**. Edit your MCP config at:

```
%APPDATA%\Claude\claude_desktop_config.json
```

Add:

```
{
  "mcpServers": {
    "everything": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-everything"]
    }
  }
}
```

Restart Claude Desktop. You should now see the "everything" server's tools available in your chat. Ask Claude: *"List your available MCP tools."*

Spend 20-30 minutes exploring. Ask Claude to use various tools, ask it to read resources, look at what's exposed. This is your hands-on understanding of what MCP servers feel like from the client side. Tomorrow you build your own.

110-120 **Capture**

Note the 5 MCP primitives in your own words, with one example each. Also note the difference between stdio and streamable-http and when to use each. Commit.

END OF DAY Claude Desktop talking to an MCP server. Five primitives memorized.

WEEK 04

Tuesday

Day 23 of 112

Your First MCP Server

INTENT Write a server with FastMCP. Decorators turn Python functions into MCP tools.

00-10 Warm-up

FastMCP makes MCP servers feel as simple as Flask. You decorate a function with `@mcp.tool()` and the protocol stuff disappears. Today: a personal notes server.

10-40 Theory — FastMCP in 30 minutes

Skim [the FastMCP docs](#). Key concepts:

- `FastMCP("server name")` — create a server instance.
- `@mcp.tool()` — decorator that turns a Python function into an MCP tool. Type annotations become the input schema automatically.
- `@mcp.resource("uri://pattern/{var}")` — exposes a resource. The URI can have path parameters.
- `@mcp.prompt()` — exposes a reusable prompt template.
- `mcp.run()` — starts the server. Default transport is stdio.

Two style notes:

- **Docstrings become tool descriptions.** Treat them with the same care you treated tool descriptions in Week 3 — five properties, when to use, when not to use.
- **Type annotations matter.** `def add_note(title: str, body: str) -> str` generates the JSON schema. Don't skip types.

+30 ▶Watch on Anthropic Academy

From the **Model Context Protocol** module ([Skilljar](#)): watch any lessons covering *building a server* or *tools*. (Module structure not screenshot-verified — adjust based on your sidebar.)

If your Skilljar module doesn't have a clear "build a server" lesson, the official tutorial at py.sdk.modelcontextprotocol.io is the canonical reference.

40-110 Lab — the notes server

Create `labs\week-04\notes_server.py`:

```
import json
from pathlib import Path
from mcp.server.fastmcp import FastMCP

# Store notes in a local file
NOTES_FILE = Path("notes_store.json")

def _load() -> dict[str, str]:
    if NOTES_FILE.exists():
        return json.loads(NOTES_FILE.read_text())
    return {}

def _save(notes: dict[str, str]) -> None:
    NOTES_FILE.write_text(json.dumps(notes, indent=2))

mcp = FastMCP("personal-notes")

@mcp.tool()
def add_note(title: str, body: str) -> str:
    """Add a new note. Use this when the user wants to save information for later.
    Do NOT use to update or delete – use update_note or delete_note instead.

    Args:
        title: Short identifier for the note. Must be unique. Lowercase, hyphens.
        body: Full content of the note. Any text, markdown allowed.

    Returns: confirmation message.
    """
    notes = _load()
    if title in notes:
        return f"Error: note '{title}' already exists. Use update_note to modify."
    notes[title] = body
    _save(notes)
    return f"Note '{title}' saved."

@mcp.tool()
def list_notes() -> list[str]:
    """List all note titles. Use to see what notes are available before reading or
    updating."""
    return sorted(_load().keys())

@mcp.tool()
def get_note(title: str) -> str:
    """Read the full content of a specific note by title.

    Args:
        title: The unique title of the note. Get titles from list_notes first if unsure.
```

```

"""
    notes = _load()
    if title not in notes:
        return f"Error: note '{title}' not found. Use list_notes to see available titles."
    return notes[title]

@mcp.resource("note://{title}")
def note_resource(title: str) -> str:
    """Expose each note as a resource. Clients that support resources can browse them."""
    notes = _load()
    return notes.get(title, f"(no note titled '{title}')")

if __name__ == "__main__":
    # Default transport is stdio
    mcp.run()

```

Test it directly with the MCP Inspector — a debugging tool that lets you call your server's tools without writing client code:

```

npx -y @modelcontextprotocol/inspector uv run python labs\week-04
otes_server.py

```

The Inspector opens in your browser. You should see your three tools (`add_note` , `list_notes` , `get_note`) and your resource. Click "Tools", select `add_note` , fill in args, hit "Call". You should see the response and the note saved to disk.

Try a few things:

1. Add 3 notes through the Inspector.
2. Call `list_notes` — should return your 3 titles.
3. Call `add_note` with a duplicate title — see the error handling work.
4. Read a resource via the "Resources" tab.

Then connect Claude Desktop to your new server. Add to `claude_desktop_config.json` :

```

{
  "mcpServers": {
    "personal-notes": {
      "command": "uv",
      "args": ["run", "python", "C:\\Users\\\\"Desktop\\cca-prep\\labs\\week-04\\
otes_server.py"],
      "cwd": "C:\\Users\\\\"Desktop\\cca-prep"
    }
  }
}

```

Restart Claude Desktop. Ask: "Save a note titled 'cca-prep-day-23' with body 'Built my first MCP server. Feels powerful.'" Then: "What notes do I have?"

110-120 Capture

Note the FastMCP pattern in one paragraph. Specifically: how docstrings become tool descriptions, how type annotations become schemas. Commit your server code.

END OF DAY A working MCP server with 3 tools and 1 resource. Claude Desktop is using it.

WEEK 04

Wednesday

Day 24 of 112

Resources & Prompts

INTENT Go beyond tools. Expose data via resources and reusable templates via prompts. Most MCP servers in the wild use all three primitives — yours should too.

00-10 Warm-up

Tools = verbs (actions). Resources = nouns (data). Prompts = reusable instructions. Today you build a server with all three, exposing a small bug tracker.

10-40 Theory — when to use which

Tools when the LLM needs to do something with side effects. Adding a note, processing a refund, sending an email. Tools mutate state.

Resources when the LLM needs to READ data. Files, documents, database records. Resources are URI-addressed and the client can choose which to include in context. Resources DON'T mutate.

Why does the distinction matter? Two reasons:

- Performance: clients can browse resources without invoking the LLM. A user can pick which docs to attach before sending a query, costing zero tokens.
- Permissions: clients can show resources to the user for approval (think file picker) in ways that wouldn't make sense for tools.

Prompts are reusable templates the server provides. A code-review MCP server might provide a "review_pull_request" prompt that takes a diff and produces a formatted review. The user picks the prompt from a list, fills in the variables, sends to the LLM. This is how server vendors share best practices.

Common exam scenario: a question describes wanting Claude to access "the company wiki." Right answer: expose wiki pages as resources, not tools. Wrong answer: write a `read_wiki_page` tool.

+30 ▶ Watch on Anthropic Academy

From the **Model Context Protocol** module ([Skilljar](#)): watch any lessons on *resources* and *prompts*. (Module lesson names not screenshot-verified.)

If you can't find direct lessons, the [FastMCP docs](#) examples cover resources and prompts thoroughly.

40-110 Lab — the bug tracker server

Create `labs\week-04\bugtracker_server.py`:

```
import json
from datetime import datetime
from pathlib import Path
from mcp.server.fastmcp import FastMCP

BUGS_FILE = Path("bugs_store.json")

def _load() -> list[dict]:
    if BUGS_FILE.exists():
        return json.loads(BUGS_FILE.read_text())
    return []

def _save(bugs: list[dict]) -> None:
    BUGS_FILE.write_text(json.dumps(bugs, indent=2))

mcp = FastMCP("bug-tracker")

# --- Tools (mutating actions) ---

@mcp.tool()
def report_bug(title: str, description: str, severity: str) -> dict:
    """Report a new bug. Use when the user describes a problem they want tracked.

    Args:
        title: Short bug title.
        description: Full reproduction steps and observed behavior.
        severity: One of "critical", "major", "minor", "nit".
    """
    if severity not in {"critical", "major", "minor", "nit"}:
        return {"error": f"Invalid severity: {severity}"}
    bugs = _load()
    bug = {
        "id": f"BUG-{len(bugs)+1:03d}",
        "title": title,
        "description": description,
        "severity": severity,
        "status": "open",
        "reported_at": datetime.now().isoformat(timespec="seconds"),
    }
    bugs.append(bug)
    _save(bugs)
    return {"created": bug["id"], "bug": bug}

@mcp.tool()
def close_bug(bug_id: str, resolution: str) -> dict:
    """Close a bug with a resolution note."""
```

```

bugs = _load()
for bug in bugs:
    if bug["id"] == bug_id:
        bug["status"] = "closed"
        bug["resolution"] = resolution
        _save(bugs)
        return {"closed": bug_id}
return {"error": f"Bug not found: {bug_id}"}

# --- Resources (read-only data) ---

@mcp.resource("bugs://open")
def open_bugs() -> str:
    """List of all open bugs, formatted as Markdown."""
    bugs = [b for b in _load() if b["status"] == "open"]
    if not bugs:
        return "# No open bugs"
    lines = ["# Open Bugs"]
    for b in sorted(bugs, key=lambda x: x["severity"]):
        lines.append(f"- **{b['id']}** [{b['severity']}] {b['title']}")
    return "\n".join(lines)

@mcp.resource("bug://{bug_id}")
def single_bug(bug_id: str) -> str:
    """Read details of a single bug by ID."""
    for b in _load():
        if b["id"] == bug_id:
            return json.dumps(b, indent=2)
    return f"(no bug with id {bug_id})"

# --- Prompts (reusable templates) ---

@mcp.prompt()
def triage_open_bugs() -> str:
    """A prompt to triage all open bugs and produce a prioritized list."""
    return (
        "Read the resource at bugs://open. For each bug, assess: "
        "(1) severity (critical/major/minor), "
        "(2) likely root cause area (UI, API, infra, data), "
        "(3) recommended owner. "
        "Output a Markdown table sorted by severity."
    )

if __name__ == "__main__":
    mcp.run()

```

Test with the Inspector:

```
npx -y @modelcontextprotocol/inspector uv run python labs\week-04\bugtracker_server.py
```

Try:

1. Report 3 bugs with different severities via the `report_bug` tool.
2. Go to the **Resources** tab. You should see `bugs://open` as a resource. Read it — the Markdown list appears.
3. Read `bug://BUG-001` as a resource — get the JSON for one bug.
4. Go to the **Prompts** tab. Select `triage_open_bugs`. The prompt text is what would be sent to the LLM.

Notice: resources show up in a different tab from tools. That's because UIs (like Claude Desktop) typically show resources in a file-picker-like UI, while tools are invoked by the LLM directly. **Same data, different access pattern.**

110-120 Capture

Note: when to use a tool vs a resource vs a prompt. One example each. This decision is exam-tested. Commit.

END OF DAY MCP server exposing all three primitives. You can articulate when each is appropriate.

WEEK 04

Thursday

Day 25 of 112

Transports — Stdio vs Streamable HTTP

INTENT Convert your local stdio server to a networked HTTP server. Learn what changes (transport) and what doesn't (your tool code).

00-10 Warm-up

So far your MCP servers run locally — Claude Desktop launches them as subprocesses. Today you run one over HTTP on localhost, the same way a production server would on the internet.

10-40 Theory — stdio vs streamable-http

The MCP spec defines two transports. They differ in deployment model:

stdio

- Server runs as a subprocess of the client.
- Communication over stdin (client → server) and stdout (server → client).
- No network. No auth (you trust the local process).
- Best for: developer tools, local utilities, anything bundled with the client.
- Examples: filesystem servers, git tools, local DB access.

streamable-http

- Server runs as a standalone HTTP service.
- Communication over HTTP POST + server-sent events (SSE) for streaming.
- Requires auth (OAuth 2.1 typical, bearer tokens, etc.).
- Best for: SaaS integrations, multi-tenant services, anything cross-organization.
- Examples: GitHub MCP, Slack MCP, your company's internal Salesforce MCP.

The good news: your tool/resource/prompt code doesn't change. Only how you call `mcp.run()` changes:

- `mcp.run()` — defaults to stdio
- `mcp.run(transport="streamable-http")` — runs as HTTP server (default localhost:8000)

Exam-relevant: a scenario asking "we want to share this MCP server with three teams in our company" — answer is streamable-http with auth, not stdio (which would require each team to install the subprocess locally).

+30 ▶ Watch on Anthropic Academy

From the **Model Context Protocol** module ([Skilljar](#)): watch any lessons covering *transports*, *HTTP*, or *deployment*. (Lesson names not screenshot-verified.)

Backup: the [MCP spec](#) transport section is short and authoritative.

40-110 Lab — your notes server, over HTTP

Copy yesterday's notes server to a new file and change the transport. Create `labs\week-04\notes_server_http.py` :

```
# Same imports + tool definitions as notes_server.py.
# The ONLY change is in __main__:

# ... all the @mcp.tool() functions exactly as before ...

if __name__ == "__main__":
    # Default port is 8000. Override with: mcp.run(transport="streamable-http", port=9000)
    mcp.run(transport="streamable-http")
```

(For brevity, copy the tool definitions from Tuesday's `notes_server.py` and just swap the final block.)

Start the server:

```
uv run python labs\week-04
otes_server_http.py
```

It should print something like *"Uvicorn running on http://127.0.0.1:8000"*. Leave it running.

In a SECOND PowerShell window, test with the Inspector pointed at the HTTP URL:

```
npx -y @modelcontextprotocol/inspector
```

In the Inspector UI, choose **"streamable-http"** as the transport and enter `http://127.0.0.1:8000/mcp` as the URL. Connect. Your tools should appear, identical to stdio mode.

Now think about auth. Right now anyone who can reach `localhost:8000` can use your server. For production, you'd add OAuth 2.1 or bearer token auth at the transport layer. The MCP spec has the auth section if you want to read deeper today; it's part of Project 2 in Week 10-11 so you'll touch this again.

Also try connecting Claude Desktop to your HTTP server. The config syntax differs slightly:

```
{
  "mcpServers": {
    "personal-notes-http": {
      "url": "http://127.0.0.1:8000/mcp"
    }
  }
}
```

Restart Claude Desktop. Your server should appear and work identically. Same code, different deployment.

110-120 Capture

Note the trade-offs: stdio vs streamable-http, and one specific scenario that fits each. Commit.

END OF DAY Your notes server running over HTTP, talking to both the Inspector and Claude Desktop.

WEEK 04

Friday

Day 26 of 112

Connecting Claude to MCP — Programmatic Client

INTENT Connect your Python agent to an MCP server programmatically (not via Claude Desktop). This is how production MCP integrations work.

00-10 Warm-up

So far Claude Desktop has been the MCP client. Today YOU write the client — a Python program that connects to an MCP server, lists its tools, calls them, and routes results into a Claude agent loop.

10-40 Theory — the MCP connector pattern

Architecturally, an MCP-aware agent looks like:

```
[user] → [your agent code] → [Claude API] (decides tool calls)
      ↓
      [MCP client] → [MCP server(s)] (executes tools)
      ↓
      [tool results] → [Claude API] (final answer)
```

The MCP client is a thin layer that:

1. Connects to one or more MCP servers (stdio or http).
2. Discovers their tools (calls `list_tools`).
3. Converts MCP tool definitions into the format Claude expects in `tools=[...]`.
4. When Claude calls a tool, routes the call to the right MCP server.
5. Returns the MCP server's response as a `tool_result`.

The `mcp` Python SDK provides `ClientSession` for this. The Anthropic Python SDK also has a higher-level `MCPClient` integration in recent versions that handles the format conversion for you.

Why bother writing your own MCP client when Claude Desktop exists? Because production agents aren't Claude Desktop. They're Python services running in containers, calling Claude API directly, needing to access company MCP servers. That's what you build today.

+30 ▶Watch on Anthropic Academy

From the **Model Context Protocol** module ([Skilljar](#)): watch any lessons covering *clients* or *integration*. (Lesson names not verified.)

Backup canonical reference: [the official Python SDK README](#) has a client example near the top.

40-110 Lab — Claude agent + your MCP server

Make sure your stdio notes server from Tuesday still exists. Create `labs\week-04\mcp_client.py`:

```
import asyncio
import json
from contextlib import AsyncExitStack
from anthropic import Anthropic
from dotenv import load_dotenv
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

load_dotenv()
client = Anthropic()

async def run_mcp_agent(user_message: str, server_script: str) -> str:
    """Connect to an MCP server, expose its tools to Claude, run an agent loop."""
    async with AsyncExitStack() as stack:
        # 1. Start the MCP server as a subprocess and open a session
        server_params = StdioServerParameters(
            command="uv",
            args=["run", "python", server_script],
        )
        read_stream, write_stream = await
stack.enter_async_context(stdio_client(server_params))
        session = await stack.enter_async_context(ClientSession(read_stream, write_stream))
        await session.initialize()

        # 2. Discover tools and convert to Anthropic format
        mcp_tools_resp = await session.list_tools()
        anthropic_tools = []
        for t in mcp_tools_resp.tools:
            anthropic_tools.append({
                "name": t.name,
                "description": t.description or "",
                "input_schema": t.inputSchema,
                "strict": True,
            })
        print(f"Discovered {len(anthropic_tools)} MCP tools: {[t[\"name\"] for t in anthropic_tools]}")

        # 3. Run the agent loop
        messages = [{"role": "user", "content": user_message}]
        for _ in range(10):
            response = client.messages.create(
                model="claude-sonnet-4-6",
                max_tokens=1024,
                tools=anthropic_tools,
                messages=messages,
            )

            if response.stop_reason == "end_turn":
                return next((b.text for b in response.content if b.type == "text"), "")
```

```

        if response.stop_reason == "tool_use":
            messages.append({"role": "assistant", "content": response.content})
            tool_results = []
            for block in response.content:
                if block.type == "tool_use":
                    print(f"  Calling MCP tool: {block.name}({block.input})")
                    # 4. Route the call to the MCP server
                    mcp_result = await session.call_tool(block.name, block.input)
                    # Extract content (mcp returns a list of content blocks)
                    content = ""
                    for c in mcp_result.content:
                        if hasattr(c, "text"):
                            content += c.text
                    tool_results.append({
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": content,
                    })
            messages.append({"role": "user", "content": tool_results})

    return "(exceeded iterations)"

async def main():
    server = "labs\\week-04\\
otes_server.py"
    for prompt in [
        "Save a note titled \"day-26-progress\" with the body \"Built an MCP client today.
Pretty neat.\",
        "What notes do I have?",
        "Read the note titled \"day-26-progress\" back to me.",
    ]:
        print(f"
User: {prompt}")
        answer = await run_mcp_agent(prompt, server)
        print(f"Claude: {answer}")

if __name__ == "__main__":
    asyncio.run(main())

```

Run:

```
uv run python labs\week-04\mcp_client.py
```

What's happening end-to-end:

1. Your client spawns the notes server as a subprocess.
2. Discovers its 3 tools via MCP protocol.
3. Converts them to Anthropic's `tools=[]` format.
4. Runs the same agent loop you've been writing all week.
5. When Claude calls a tool, your client routes the call to the MCP server, gets the result, sends it back to Claude.

This is the production pattern. Project 2 (Week 10-11) uses this exact architecture for the Customer Support Agent.

110-120 **Capture**

Draw the MCP-aware agent architecture (your code, Claude API, MCP client, MCP server) on paper. Save the image or describe in notes. This diagram appears on the exam. Commit.

END OF DAY Python agent talking to your own MCP server end-to-end. No Claude Desktop required.

WEEK 04

Saturday

Day 27 of 112

Multi-Server & Production Concerns

INTENT Connect to multiple MCP servers at once. Touch authentication, scopes, and the production deployment story.

00-10 Warm-up

Real agents talk to MULTIPLE MCP servers — one for the company wiki, one for tickets, one for the CRM. Today you wire two together.

10-40 Theory — multi-server, scopes, auth

Multi-server: your client can connect to several MCP servers in one session. You merge their tools into one big `tools=[]` array. When Claude picks a tool by name, you route to the correct server based on which session owns it.

Naming becomes critical: if both servers expose `get_user`, Claude can't tell them apart. Conventions:

- Prefix tool names by server: `crm_get_user`, `helpdesk_get_user`
- Or use scoped names in the server design from the start

Authentication for remote servers: The MCP spec defines OAuth 2.1 as the standard for remote (streamable-http) servers. Flow:

1. Client requests access from the auth provider.
2. User approves scopes (e.g. "read tickets", "create tickets", "delete tickets").
3. Client receives a token, passes it as `Authorization: Bearer ...` on every MCP call.
4. Server validates the token and enforces the scopes per tool/resource.

Scopes are tool-level. A "ticket reader" token can call `list_tickets` but is denied `close_ticket`. The server enforces this — your client just passes the token through.

Exam-relevant: "How do you give Claude access to a customer's private MCP server without exposing it publicly?" → OAuth 2.1 flow with user-scoped tokens, scopes per tool.

+30 ►Watch on Anthropic Academy

From the **Model Context Protocol** module ([Skilljar](#)): watch any lessons on *authentication*, *OAuth*, or *production deployment*. (Lesson names not verified.)

If the Skilljar module doesn't cover OAuth deeply, the [MCP spec authorization section](#) is the authoritative source.

40-110 Lab — multi-server agent

Create `labs\week-04\multi_mcp_client.py`:

```
import asyncio
from contextlib import AsyncExitStack
from anthropic import Anthropic
from dotenv import load_dotenv
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

load_dotenv()
client = Anthropic()

async def connect_to_server(stack, name: str, script: str):
    """Open an MCP session to a stdio server."""
    params = StdioServerParameters(command="uv", args=["run", "python", script])
    read, write = await stack.enter_async_context(stdio_client(params))
    session = await stack.enter_async_context(ClientSession(read, write))
    await session.initialize()
    return session

def prefix_tool_name(server_name: str, tool_name: str) -> str:
    return f"{server_name}__{tool_name}"

async def run_multi_mcp_agent(user_message: str) -> str:
    async with AsyncExitStack() as stack:
        # Connect to both servers
        notes = await connect_to_server(stack, "notes", "labs\\week-04\
otes_server.py")
        bugs = await connect_to_server(stack, "bugs", "labs\\week-04\\bugtracker_server.py")

        sessions = {"notes": notes, "bugs": bugs}

        # Collect tools from all servers, prefixed by server name
        anthropic_tools = []
        tool_routing: dict[str, tuple[str, str]] = {} # prefixed name -> (server_name,
original_name)

        for server_name, session in sessions.items():
            resp = await session.list_tools()
            for t in resp.tools:
                prefixed = prefix_tool_name(server_name, t.name)
                anthropic_tools.append({
                    "name": prefixed,
                    "description": f"[{server_name}] {t.description or \"\"}",
                    "input_schema": t.inputSchema,
                    "strict": True,
                })
                tool_routing[prefixed] = (server_name, t.name)

        print(f"Total tools across {len(sessions)} servers: {len(anthropic_tools)}")
```

```

# Standard agent loop, with routing
messages = [{"role": "user", "content": user_message}]
for _ in range(10):
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=1024,
        tools=anthropic_tools,
        messages=messages,
    )

    if response.stop_reason == "end_turn":
        return next((b.text for b in response.content if b.type == "text"), "")

    if response.stop_reason == "tool_use":
        messages.append({"role": "assistant", "content": response.content})
        tool_results = []
        for block in response.content:
            if block.type == "tool_use":
                server_name, original_name = tool_routing[block.name]
                session = sessions[server_name]
                print(f"  Routing {block.name} -> {server_name}.{original_name}")
                result = await session.call_tool(original_name, block.input)
                content = "".join(c.text for c in result.content if hasattr(c,
"text"))

                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": content,
                })
            messages.append({"role": "user", "content": tool_results})

        return "(exceeded iterations)"

async def main():
    prompt = (
        "Save a note titled \"thursday-bug\" describing that the login button is broken on
mobile. "
        "Then report this as a bug with severity major. "
        "Finally, list all my open bugs."
    )
    answer = await run_multi_mcp_agent(prompt)
    print(f"
Final answer:
{answer}")

if __name__ == "__main__":
    asyncio.run(main())

```

Run it. Claude should:

1. Use `notes__add_note` (routes to notes server)

-
2. Use `bugs__report_bug` (routes to bug tracker)
 3. Use `bugs__open_bugs` via a resource read, or list via a tool call

Watch the routing log to confirm tools go to the right server.

What you've just built is the architecture of every real MCP-using agent. Project 2's customer support agent in Week 10-11 will connect to your custom MCP server using exactly this pattern.

110-120 **Capture**

Note: the multi-server pattern (prefix names, route by prefix). Also note one OAuth scenario in your own words. Commit.

END OF DAY Multi-server MCP agent. Routing works correctly. OAuth 2.1 understood at concept level.

WEEK 04

Sunday

Day 28 of 112

Week 4 Review & Project 1 Prep

INTENT Lock in MCP. Take the quiz. Get psychologically ready for Week 5 — your first portfolio project.

00–10 Warm-up

End of foundations phase. 28 days, 4 weeks, dozens of commits. Tomorrow you start a portfolio project that recruiters and exam graders will actually look at.

10–60 Review — re-run every Week 4 lab

```
uv run python labs\week-04
otes_server.py          # in one window, then Ctrl+C
uv run python labs\week-04\bugtracker_server.py      # in another, then Ctrl+C
uv run python labs\week-04
otes_server_http.py     # HTTP version
uv run python labs\week-04\mcp_client.py             # programmatic client
uv run python labs\week-04\multi_mcp_client.py      # multi-server
```

For each, ask: "Could I rewrite this from scratch tomorrow?"

+30 ►Watch on Anthropic Academy

Formal self-test for Week 4 ([Skilljar](#) • *Model Context Protocol* module):

- End-of-module quiz on MCP (look for `Quiz on Model Context Protocol` or similar at the end of the module's lesson list)

Aim 80%+.

60-110 Self-check, then Project 1 brief

Week 4 self-check question (from the master plan): *"Stdio vs streamable HTTP — when do you pick each? What auth model does each support?"*

Write the answer in `notes\weekly-reviews.md`. Then write:

1. Hardest moment this week was...
2. The MCP concept I'm shakiest on is...
3. What I'm most excited about for Project 1 is...

Project 1 setup — read the spec in your master plan PDF, Section 4. The next two weeks (Days 29-42) build the **Structured Data Extractor** — a system that extracts typed JSON from messy invoices/contracts/receipts with measurable accuracy. You already know every primitive needed:

- Vision API for image-mode PDFs (Day 10 W2)
- Structured outputs with Pydantic (Days 10-11 W2, post-errata)
- Validation retry loop for semantic failures (Day 12 W2)
- Prompt caching for cost (Day 13 W2)
- Tool use for confidence routing (Week 3)

Create the project repo:

```
cd ..
mkdir project-01-extractor
cd project-01-extractor
gh repo create cca-prep-project-01-extractor --private
uv init
uv add anthropic pydantic typer rich pytest python-dotenv pypdf
```

Find a public invoice dataset on Kaggle or Hugging Face. Download ~10 samples to start (you'll hand-label 50 by mid-Week-5). Save to `data/raw/`.

That's enough prep for today. Tomorrow you start building.

110-120 Capture

```
cd ..\cca-prep
git add .
git commit -m "week 4 complete: MCP"
git push
```

Close the laptop. Rest tonight — Week 5 is the start of your first real ship.

END OF DAY Week 4 complete. Foundations phase done. Project 1 repo initialized.